The "goto" module was an April Fool's joke, published on 1st April 2004. Yes, it works, but it's a joke nevertheless. Please don't use it in real code!

Also available in Russian (thanks, Ruslan)

goto for Python

Adds the 'goto' and 'comefrom' keywords to Python.

The 'goto' and 'comefrom' keywords add flexibility to Python's control flow mechanisms, and allow Python programmers to use many common control flow idioms that were previously denied to them. Some examples are given below.

To enable the new keywords, add the following line to the top of your code:

```
from goto import goto, comefrom, label
```

goto

'goto' jumps the program execution directly to another line of code. The target line must be identified using a 'label' statement. Labels are defined using the 'label' keyword, and have names which are arbitrary Python identifiers prefixed with a single dot, like this:

```
label .myLabel
```

To jump to a label, use the 'goto' keyword like this:

```
goto .myLabel
```

Computed goto

You can use a computed goto by assigning the name of the label to a variable at runtime and referencing it using an asterisk like this:

```
x = calculateLabelName()
goto *x
```

The value of 'x' should not include the leading dot. See Example 5 below for a full example.

comefrom

'comefrom' is the opposite of 'goto'. It allows a piece of code to say "Whenever label X is reached, jump to here instead." For example:

```
# ...code 1...
label .somewhere
# ...code 2...
comefrom .somewhere
```

Here, "code 2" will not run - execution will jump directly from the "label .somewhere" line to the "comefrom .somewhere" line. 'comefrom' is typically used as a debugging aid - its use in production code is discouraged since it can lead to surprising behaviour.

Restrictions

There are some classes of goto and comefrom which would be unpythonic, and hence there are some restrictions on where jumps can go:

- No jumping between modules or functions
- No jumping into the middle of a loop or a 'finally' clause
- No jumping onto an 'except' line (because there is no exception)

Examples

Here are some examples of how goto and comefrom can be used:

```
# Example 1: Breaking out from a deeply nested loop:
from goto import goto, label
for i in range(1, 10):
    for j in range(1, 20):
        for k in range(1, 30):
            print i, j, k
            if k == 3:
                goto .end
label .end
print "Finished\n"


# Example 2: Restarting a loop:
from goto import goto, label
label .start
for i in range(1, 4):
    print i
    if i == 2:
        try:
            output = message
        except NameError:
            print "Oops - forgot to define 'message'!  Start again."
            message = "Hello world"
            goto .start
print output, "\n"


# Example 3: Cleaning up after something fails:
from goto import goto, label

# Imagine that these are real worker functions.
def setUp(): print "setUp"
def doFirstTask(): print 1; return True
def doSecondTask(): print 2; return True
def doThirdTask(): print 3; return False  # This one pretends to fail.
def doFourthTask(): print 4; return True
```

```
def cleanUp(): print "cleanUp"

# This prints "setUp, 1, 2, 3, cleanUp" - no "4" because doThirdTask fails.
def bigFunction1():
    setUp()
    if not doFirstTask():
        goto .cleanup
    if not doSecondTask():
        goto .cleanup
    if not doThirdTask():
        goto .cleanup
    if not doFourthTask():
        goto .cleanup

    label .cleanup
    cleanUp()

bigFunction1()
print "bigFunction1 done\n"


# Example 4: Using comefrom to let the cleanup code take control itself.
from goto import comefrom, label
def bigFunction2():
    setUp()
    if not doFirstTask():
        label .failed
    if not doSecondTask():
        label .failed
    if not doThirdTask():
        label .failed
    if not doFourthTask():
        label .failed

    comefrom .failed
    cleanUp()

bigFunction2()
print "bigFunction2 done\n"


# Example 5: Using a computed goto:
from goto import goto, label

label .getinput
i = raw_input("Enter either 'a', 'b' or 'c', or any other letter to quit: ")
if i in ('a', 'b', 'c'):
    goto *i
else:
    goto .quit

label .a
print "You typed 'a'"
goto .getinput

label .b
print "You typed 'b'"
```

```
goto .getinput

label .c
print "You typed 'c'"
goto .getinput

label .quit
print "Finished\n"


# Example 6: What happens when a label is missing:
from goto import goto, label
label .real
goto .unreal      # Raises a MissingLabelError exception.
```

This module is released under the Python Software Foundation license, which can be found at http://www.python.org/ It requires Python 2.3 or later.

Richie Hindle, richie@entrian.com

Version 1.0, released 1st April 2004. Download here.